

Course Proposal: AI-Accelerated Software Development

Tagline: Helping small teams build like big teams.

Concept

Hennepin Tech can offer a practical, hands-on AI software development course for local businesses that need more development capacity than their current team size allows. This is not a general AI awareness class. It is a working course where participants use modern AI coding agents to plan, build, test, debug, review, and improve real software.

It is also different from a traditional "learn to code with AI" class. That is a valid and important direction, especially for people who want to become stronger programmers. This course is aimed at a different need: smaller teams with big business ideas, where the code matters but the business capability matters more. Participants learn how to write and modify business applications with AI agents: define the capability, direct the work, inspect the result, test the workflow, and decide whether it is good enough for business use.

The course is built from the AI Development Curriculum in this repository. Students learn a repeatable development workflow:

intention -> requirements -> implementation plan -> agent-assisted build -> browser verification -> quality scan -> review

That workflow gives small teams a disciplined way to use AI tools without turning software delivery into trial-and-error prompting.

Audience

This flagship course is designed for:

- Software developers who build or maintain applications.
- Senior developers and technical leads responsible for review and delivery.
- Engineering managers who remain hands-on in code and technical decisions.
- Platform, QA-automation, and DevOps engineers who can work in the repository.

Participants must be able to read and modify code, use Git and a terminal, and understand tests, dependencies, pull requests, and debugging. A separate one- or two-day offering can be developed for managers, analysts, operations, and other non-developer staff; they are not the audience for this five-day lab.

Each participant needs client-approved access to Cursor, Claude Code, or Codex with enough model capacity for the labs. Cursor is the recommended common IDE; Claude Code and Codex are fully supported. Exact plans, pricing, model access, and data terms are confirmed before each cohort.

The application tracks are adaptable to the client. In particular, the Twitter and Facebook examples have supplied native iOS references, but neither effort is required to remain iOS: a cohort may deliver either as a web application. Web is the cross-platform teaching default; SwiftUI/Xcode is an optional Mac-only specialization.

Business Problem

Many local businesses need custom software faster than their teams can deliver: internal tools, customer-facing applications, workflow automation, dashboards, integrations, reporting, product prototypes, and revenue-generating software experiments. Hiring more developers is expensive and slow. Buying commercial software often creates a poor fit, recurring costs, or fragmented workflows.

AI coding agents give small teams more leverage, but only when teams know how to direct them, review their work, verify behavior, and protect business data. Without that structure, teams risk producing fragile prototypes, security gaps, unreviewed code, and false confidence.

This course teaches a controlled, practical operating model for AI-assisted development.

Why This Matters Now

Traditional software delivery was never mostly about typing code. It was about latency: requirements meetings, a prototype, a review meeting, change requests, a wait for the next sprint, and around again - with days or weeks of coordination baked into every turn of the loop. That loop is why a modest internal tool could take a month, and why so many useful ideas sit permanently in the backlog.

AI-assisted development collapses that loop for well-scoped work. When an agent can build a small, scoped slice of an application in under an hour, the expensive middle of that iteration disappears, and define and prove bracket the build - defining clearly what should exist, and validating that what got built is actually correct and safe. A full product is many such bracketed iterations; each effort should be scoped so both brackets are appropriate. A small group - a stakeholder, a couple of developers, and a tester - can sit in one room, define a capability, build it, react when the slice is ready, and iterate again before lunch. A month of meetings and hand-offs becomes a morning - for scoped slices, not for every system they'll ever ship.

That shift changes a company's whole relationship to its backlog. Work that used to be permanently postponed becomes realistic, and a business can responsibly test new products, revenue streams, and customer experiences without waiting months for a traditional software cycle to begin.

This course teaches teams how to turn that speed into a repeatable, disciplined habit: choose the right problem, define the capability clearly, use AI to build or modify the application, and verify that the result actually works - moving fast while keeping human judgment in charge.

What Makes This Offering Different

- Hands-on software delivery: Students build and test actual applications, not just watch AI demos.
- Application capability, not coding trivia: Traditional coding education still matters. This course has a different emphasis: creating and modifying business applications with AI, where success is measured by useful capability rather than lines of code.
- A different relationship to the backlog: Faster delivery makes previously unreachable software ideas - new services, internal capabilities, and revenue experiments - practical enough to prototype and learn from.
- Agentic workflow: Students learn how to direct coding agents through scoped tasks, review loops, and verification steps.
- Harness-portable: Cursor is the recommended common IDE; Claude Code and

Codex are fully supported. The course preserves one canonical knowledge and capability system while teaching native invocation, permissions, models, browser verification, and parallel work where each approved tool supports them.

- Quality and governance: Students learn how to scan for errors, test with browser automation, review code changes, and keep humans responsible for decisions.
- Business relevance: The labs map to realistic small-team needs: internal apps, external tools, customer-facing products, dashboards, SaaS-style workflows, QA, documentation, and model evaluation.

Example Applications And Workflows

The course uses realistic business application examples so participants can see how the workflow applies to ordinary company needs. Depending on cohort goals and technical level, participants may build or modify examples such as:

- Team wiki or internal knowledge base.
- Pipeline CRM or sales workflow tracker.

- Operations dashboard with status checks and incident notes.
- API workspace for testing service requests.
- Invoice, expense, or lightweight accounting workflow.
- Kanban-style project board.
- Customer portal, intake form, or self-service workflow.
- New product prototype or revenue-stream experiment.
- Browser-tested SaaS-style application with authentication and persisted data.
- Codebase research and improvement workflow for an existing application.

These are suggested applications, not a rigid list. If a business has backlog items that are a better fit, the same lessons can be adapted toward the kinds of internal applications, external tools, customer-facing products, workflow tools, reports, dashboards, or integrations they actually need written.

The second half of the week builds the three largest strategic examples as core content - not optional extras:

- Agent-to-agent protocol dashboard (Day 3): a multi-agent application where agents discover each other, delegate work, and stream the result to a dashboard.
- Skills library or skills browser (Day 3): a searchable library of reusable AI skills and workflows that a team can adapt across projects.
- LLM/model comparison lab (Day 4): an evaluation tool that compares multiple language models on realistic prompts, scoring quality, speed, and fit for the organization's work.

Any of the three can also be extended as a Day 5 final project for teams that want to go deeper.

Recommended Offerings

Flagship Course: Five-Day Intensive

AI-Accelerated Software Development: Helping Small Teams Build Like Big Teams

A week-long course for technical staff who want to use AI coding agents in real development work. This is the strongest match for a \$1,500-per-person workforce training product.

The standard version uses curriculum projects and realistic business app examples. The suggested applications can be swapped for backlog-relevant apps when a company has a clear need. Every team leaves with a started AI Program: a standard folder-backed governance and GRC structure; selectable Full, Essential, and Light views over one canonical record; AI-culture, workforce, learning, psychological-safety, sentiment-privacy, and outcome guidance; initial company-specific decisions, owners, sources, controls, evidence links, and gaps; maintained AI skills and agents; and an AI Governance & GRC application for navigating and asking questions of the program. Eleven adoption worksheets guide the intake work but are not a separate product. On Day Five, teams install and adapt the program in a current repository, a new monorepo, or a standalone company-wide AI Program repository while completing one risk-classified, verified change.

Length and scheduling (for discussion): The course is about 30 contact hours - five sessions of roughly six instructional hours each. Pulling a small team out for five consecutive business days is often impractical, so the same content can be delivered on whichever calendar fits the cohort:

- One session per week for five weeks (least disruptive for a working team; participants apply the workflow between sessions).
- Ten half-day sessions over two weeks (leaves the rest of each day for regular work).
- Five consecutive days (a traditional immersive week; best for open enrollment).

We would recommend settling the format with Hennepin Tech per cohort. Once a format is chosen, we will finalize the syllabus and marketing materials to match the selected schedule.

One-Day Executive Workshop

For owners, managers, and technical leads who need to understand the opportunity, risk, governance model, and ROI of AI-assisted software delivery.

The centerpiece is a live build loop, not slides. With the room, we define a small internal tool, build the scoped slice in about twenty minutes, take a change request live ("no, not that, more like this"), iterate, and finish before the break - so decision-makers watch a month of traditional meetings-and-hand-offs collapse into a single morning for work scoped small enough to finish in the room. The rest of the day covers where this fits, where it does not, and how to adopt it responsibly (data, security, review, and human judgment).

Half-Day Executive Demonstration and Discovery

A shorter session - roughly three hours, or half a day - that doubles as a working pitch meeting. It has two jobs at once: show decision-makers the live build loop on a problem drawn from their world, and preview exactly what the full course would build and teach for their development group. In practice we:

- Run the live build loop on a small tool relevant to the company, so leaders see "a month in a morning" for scoped work firsthand.
- Walk through what a cohort of their developers would build and learn across the five days.
- Discuss where AI-assisted delivery fits their stack, their backlog, and their risk and governance needs.

Because it also functions as discovery, it produces a tailored recommendation for a follow-on cohort. It can stand alone, but it is most valuable as the on-ramp to the full course: run the demonstration first, then deliver the five-day intensive or a private custom cohort for the team. The two are naturally sold together - the demonstration earns the room, and the course delivers the capability.

Two-Day Practitioner Bootcamp

A compressed hands-on version focused on the core workflow: planning, agent-assisted implementation, testing, and review.

Custom Onsite Course

A tailored version using the company's stack, internal workflow, or a sanitized version of one of their software problems.

This can include one or two additional setup days focused on the company's own codebase. Those days can cover repository orientation, tool installation, AI-agent permissions, local development startup, browser testing setup, team prompt/checklist creation, and selection of first low-risk backlog items.

To be determined with Hennepin Tech: Only the five-day intensive has a proposed length (~30 contact hours) and price point (~\$1,500 per person) so far. We would still need to work out the length and cost for the shorter offerings - the Half-Day Executive Demonstration, One-Day Executive Workshop, Two-Day Practitioner Bootcamp, and Custom Onsite Course - including whether the demonstration is priced on its own, credited toward a follow-on cohort, or bundled with it. We should also confirm whether Hennepin Tech offers formats like these at all. If shorter or custom formats are not something Hennepin Tech supports, we can focus the engagement on the flagship five-day course.

Bring Your Own Codebase Option

For businesses that want to apply the course directly to their own environment, Hennepin Tech can offer an optional codebase setup engagement after or alongside the class.

This option is supported by Matthew Weber (GolferGeek)'s 25 years of experience working in large codebases. That background matters because the hard part is not just installing an AI tool. The hard part is understanding an existing system, finding safe entry points, respecting architecture, and choosing backlog work that can be attempted without putting production code at risk.

The workflow itself is not theoretical: the instructor builds and maintains production-scale AI platforms - including OrchestratorAI and Divinr.ai, codebases of thousands of source files - as a solo developer using the exact pipeline this course teaches, running multiple AI harnesses in parallel on the same products. Most of that work is open source, so

prospective cohorts can inspect the results rather than take the claim on faith. The course is the packaged version of a working daily practice.

In this format, the instructor helps the team:

- Open and understand their existing repository structure.
- Configure AI development tools for local use.
- Establish safe permissions and data-handling rules.
- Document how to start the application locally.
- Set up browser-based testing or verification where appropriate.
- Create a team workflow for using AI agents responsibly.
- Identify a short list of backlog items that are good first candidates for AI-assisted implementation.

This is best offered as a private company session because real repositories may include proprietary code, customer data, internal systems, or access credentials.

Keeping Current

AI development tools are changing quickly. The course will be taught against current best practices at the time of delivery, including current guidance for tool setup, permissions, browser verification, computer-use workflows, and review discipline.

Those practices will continue to evolve. After the course, companies should plan to stay current themselves by reviewing tool documentation, updating internal policies, and revisiting their AI workflow as the tools change. If desired, they can also schedule occasional short consulting check-ins to refresh the setup or review new capabilities. The intent is not to create a long-term dependency, but to help teams get started correctly and know how to keep adapting.

Flagship Course Outcomes

By the end of the five-day course, participants will be able to:

- Turn a business idea into an intention, product requirements document, and implementation plan.
- Use AI coding agents to build or modify a working application - including apps with real authentication and per-user data.
- Configure shared project instructions for their chosen tool, and distinguish team knowledge (committed to the repo) from personal memory.
- Review AI-generated code for correctness, scope, and maintainability.
- Use browser-based testing to verify real user flows.
- Use in-app browser and computer-use workflows to inspect UI behavior, console errors, screenshots, and local development servers.
- Run build, lint, and test checks before accepting changes, and grow living review standards that prevent repeat mistakes.
- Research an unfamiliar or existing codebase (orientation, mapping, security scan, git history) before changing it.
- Describe agent-to-agent protocols at a high level: how agents discover, authorize, pay, and report to each other.
- Package organizational knowledge as canonical skills and specialized agents,

organized by function and generated for Claude Code, Cursor, and Codex.

- Query and maintain an agent-readable AI program covering models/harnesses, coding governance, protocols, security/data, delivery/quality, and adoption.
- Build a started GRC control map that distinguishes authoritative client

requirements from course defaults and connects risk tiers, agent authority, controls, evidence, rollback, monitoring, stop conditions, exceptions, accountable owners, and review triggers.

- Choose models empirically by evaluating them against the organization's own prompts, and draft a routing policy that matches model cost to task difficulty.
- Identify common AI-assisted development risks around data privacy, security, dependencies, and over-trusting generated code.
- Leave with a started adoption kit: a repeatable workflow and operating-model templates their organization can fill in and use after the course.

Sample Business Pitch

A month of software delivery was never a month of typing - it was meetings, prototypes, review cycles, and hand-offs, with a wait baked into every turn. When an agent builds a well-scoped application slice in under an hour, that loop collapses: a stakeholder, a couple of developers, and a tester can define a tool in one room, build it, react to it, and iterate before lunch. For scoped work, a month of coordination can become a morning. Larger systems still take longer - the acceleration is real, but scoping is part of the skill. Small teams are suddenly able to deliver software like much larger organizations - but only if they know how to steer the agents, verify the result, and keep humans in control.

This course teaches that operating model. Participants learn a structured workflow for using the AI coding agent of their choice to plan features, build working applications, debug issues, test in a browser, and review changes before they ship. The result is practical AI leverage for teams that need more delivery capacity without immediately adding more headcount.

Just as important, the course reframes software work around business capability. The goal is not to celebrate lines of code. The goal is to help teams finally move the dashboards, automations, integrations, customer-facing tools, new product ideas, and internal applications that have been stuck in the backlog.

The bigger shift is that the backlog itself can change. Once teams move faster, they can prototype a new workflow, customer-facing tool, or revenue idea quickly enough to learn whether it deserves more investment - and sometimes move from prototype to production far faster than older delivery assumptions would suggest.

For companies that want direct help applying the workflow, an optional setup day can focus on their own codebase: getting the tools installed, documenting the local development process, setting up browser verification, and choosing the first safe backlog items to tackle. This is backed by Matthew Weber's 25 years of experience working in large codebases, where successful change depends on understanding the existing system before modifying it.

Suggested Pricing Position

The five-day intensive can be positioned competitively against week-long technical training courses in the \$1,500-per-person range. The value proposition is stronger when framed around productivity, software delivery, and workforce readiness rather than generic AI literacy.

Publication Note

Tool names, plans, availability, provider access, and data terms should be checked before each cohort. Cursor is recommended and Claude Code/Codex are supported, but native agent, browser-control, computer-use, and model-routing features change quickly.